Project Description

High-Quality Software Construction

1. The Core Objective

The goal of this project is to design and build a complete desktop application of your own conception.

The primary focus is not the novelty or complexity of the application, but the **quality of its construction**. Your grade will be determined by your ability to apply the principles from this course to create software that is demonstrably **safe from bugs, easy to understand, and ready for change.**

2. The First Task: Conceiving Your Application

Your first and most important task is to decide what you will build. To ensure your idea is suitable for this project, it must have a clear and meaningful:

- **Internal State (The Model):** The application must manage some form of data or state that changes over time based on user actions.
- **User Interaction (The Controller):** The user must be able to perform actions that modify the application's internal state.
- **Visual Representation (The View):** The application's state must be presented visually to the user, and this view must update when the state changes.

Warning on Scope: Be realistic. A smaller, well-defined application that is flawlessly executing is far more valuable than an overly ambitious one that is incomplete and poorly constructed. Therefore, I suggest one class in each module (i.e., model, view, controller) is enough.

Phase 1 (requirements)

3. Project Proposal

Before you begin coding, you must submit the project concept and requirements in requirements section in the project template. It should contain the following:

1. **Application Concept:** A brief (1-2 paragraph) description of the application you intend to build.
2. **Functional Requirements**: Define the specific features and capabilities the application must have. This section describes what the system will do. List at least 6 requirements.

This section ensures your chosen application is a good fit for the project's learning goals.

4. Non-Negotiable Technical Requirements

Regardless of the application you choose to build, your project **must** adhere to the following technical and design constraints.

Phase 2 (design and implementation)

Technical and design requirements	Description of integration into your application
Architectural Pattern	Your application **must** be built using the **Model-View-Controller (MVC)** pattern. You must maintain a strict separation between your data/logic (Model), your UI (View), and your user-input handling (Controller).
Design Patterns	You must correctly implement **at least one** of the following design patterns as a central part of your architecture: **Observer**, **Facade**, or **Singleton**. You must justify your choice in your documentation.
Abstract Data Types (ADTs)	You must design, specify, and implement at least **one** immutable ADT that is central to your application's model. you must document **Representation Invariant (RI)** and **Abstraction Function (AF) for this ADT.**
Equality	You **must** correctly implement the `equals()` and `hashCode()` methods for at least one of your ADTs, with a clear justification for why it is necessary for your design (e.g., for use in a `Set` or as a `Map` key).

In summary, this project challenges you to practice constructing high-quality software that is safe from bugs, easy to understand, and ready for change.

Dr. Mawal A. Mohammed

Good Luck!